



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

StanaGO - Aplicație pentru comerț mioritic

Ingineria Sistemelor

Autori: Popescu Florian și Todi Tinu-Constantin
Grupa: 30237

FACULTATEA DE AUTOMATICĂ
ȘI CALCULATOARE

13 Ianuarie 2026

Cuprins

1	Introducere	2
2	Limbajele și tehnologiile folosite	2
3	Cerințe funcționale și Diagrama Use-Case	2
4	Cerințe non-funcționale	3
5	Diagrame de Stare	4
6	Design Patterns	5
6.1	Facade Pattern	5
6.2	Observer Pattern	5

1 Introducere

StanaGO este o platformă web dedicată digitalizării comerțului mioritic, având ca scop principal conectarea producătorilor locali (ciobani) cu consumatorii de produse tradiționale, cu entuziaștii drumețiilor montane sau cu cei în trecere cu mașina.

Aplicația răspunde la două nevoi majore:

- **Promovarea produselor locale:** Utilizatorii pot localiza pe hartă stânele din apropiere și pot vizualiza disponibilitatea produselor (brânză, lapte, lână).
- **Siguranța stânelor:** Printr-un sistem de raportare de pericole, ciobanii pot raporta în timp real prezența prădătorilor (urși, lupi), generând alerte vizibile pe hartă pentru a proteja stânele ciobanilor.

2 Limbajele și tehnologiile folosite

Proiectul utilizează o arhitectură "Full-Stack" bazată pe ecosistemul Microsoft .NET:

- **Backend:** C#, ASP.NET Core MVC (pentru logica aplicației și API).
- **Bază de date:** Microsoft SQL Server (stocare relațională).
- **Frontend:** HTML5, CSS3, Razor Pages.
- **Hărți & Geolocation:** Leaflet.js și OpenStreetMap (pentru randarea hărților).

3 Cerințe funcționale și Diagrama Use-Case

Funcționalitățile principale ale sistemului sunt:

1. **Înregistrare și Autentificare:** Crearea de conturi cu roluri distincte (Utilizator "Casual" sau "Cioban").
2. **Personalizare Profil:** Utilizatorii, își pot personaliza profilul și vedea și alte profile.
3. **Gestiunea Stânelor:** Ciobanii pot adăuga stâne pe hartă folosind coordonate GPS și pot administra inventarul de produse.
4. **Sistem de Alertă:** Raportarea pericolelor și afișarea lor pe hartă pe baza unei raze de proximitate.
5. **Mesagerie (Chat):** Comunicare în timp real între clienți și ciobani pentru detalii despre produse.
6. **Filtrare Geospațială:** Algoritm backend care afișează doar entitățile (stâne/alerte) pe o rază de 20 de kilometri față de utilizator.

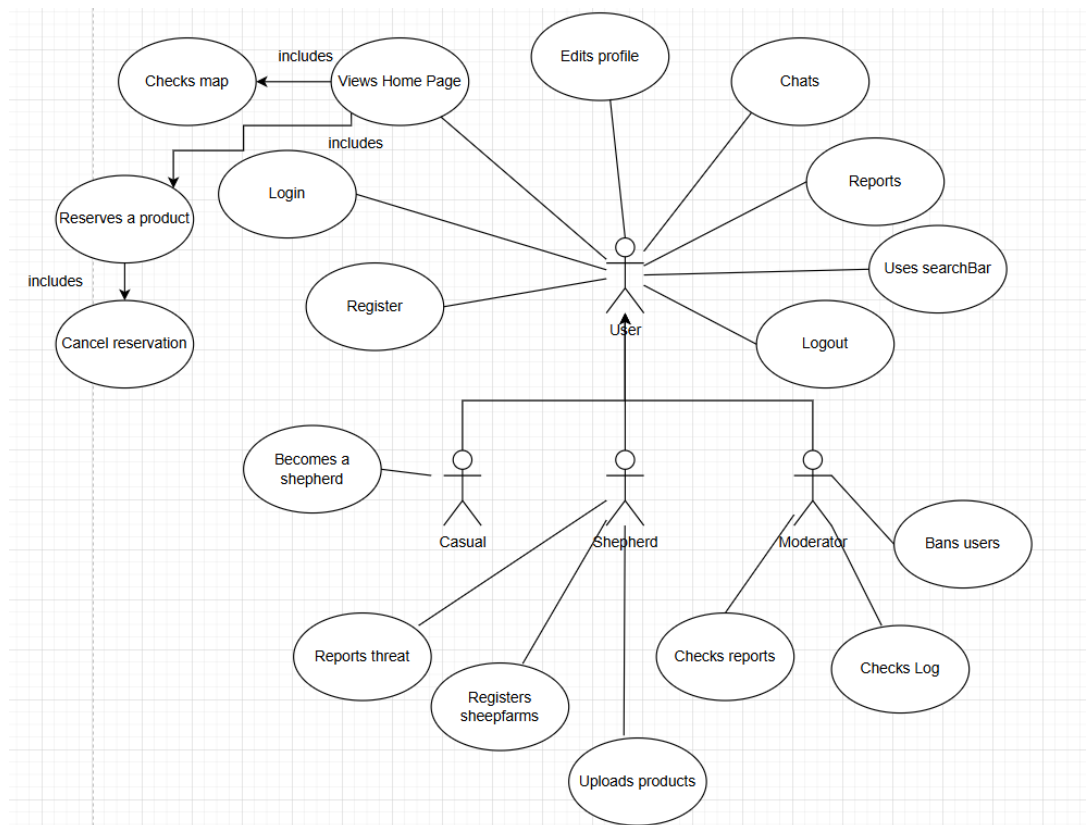


Figura 1: Diagrama Use-Case a aplicației StanaGO

4 Cerințe non-funcționale

- **Securitate:** Accesul la funcții sensibile (ex: raportare pericol) este restricționat pe bază de roluri (Role-Based Access Control). Parolele sunt stocate criptat.
- **Performanță:** Încărcarea asincronă a datelor pentru chat și hartă, reducând timpul de răspuns.
- **Portabilitate:** Interfața este "Responsive", adaptându-se automat pentru dispozitive mobile (telefoane/tablete).

5 Diagrame de Stare

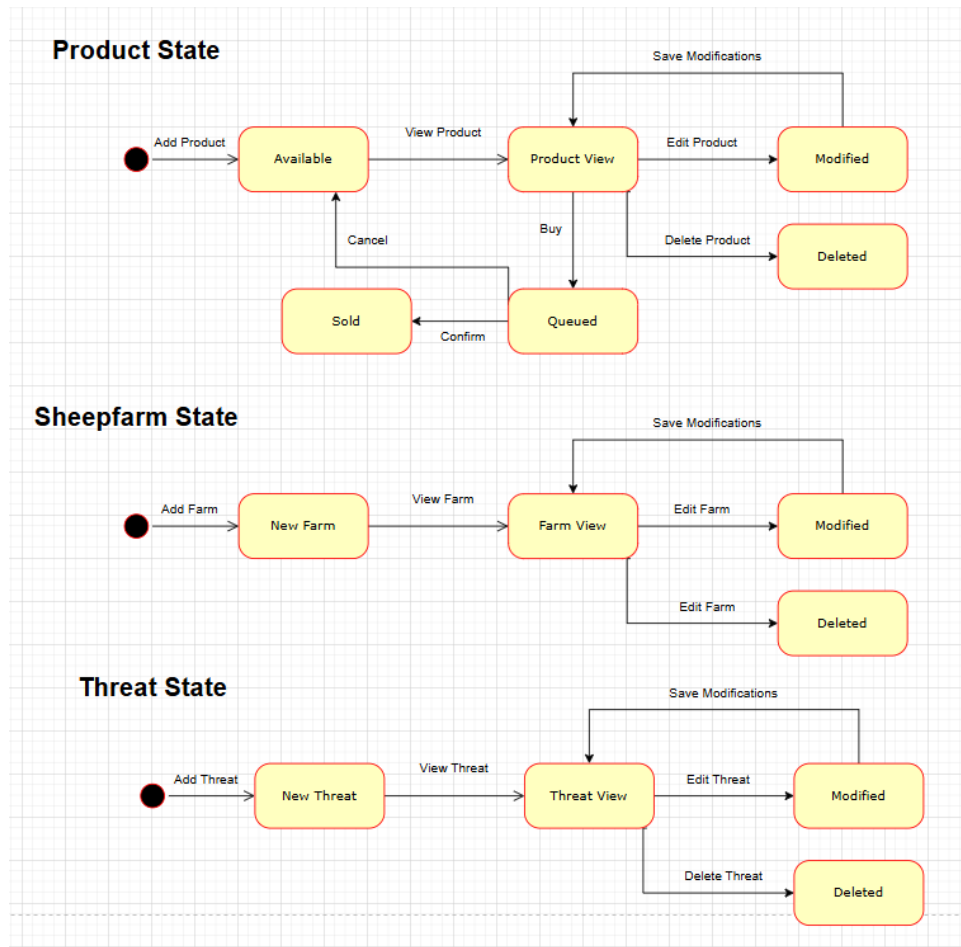


Figura 2: Diagrame de stare: Product, Sheepfarm, Threat - Popescu Florian

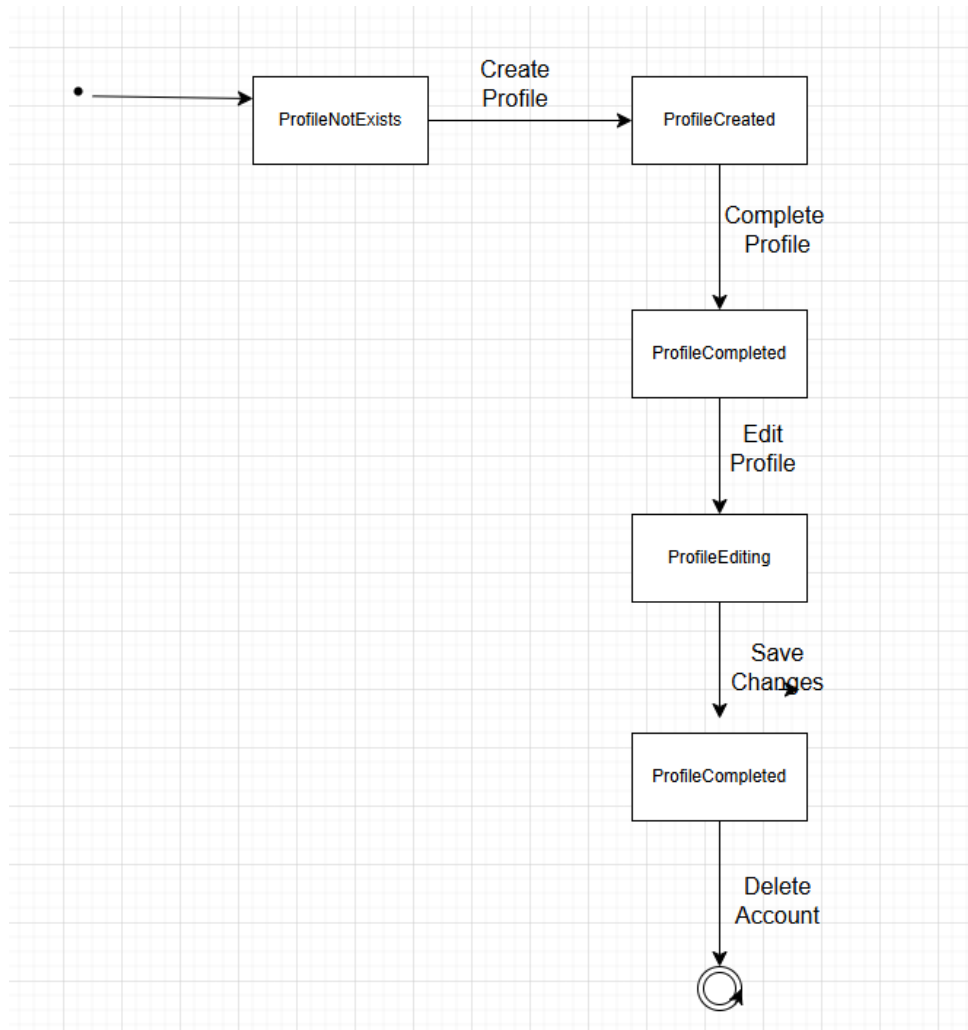


Figura 3: Diagrame de stare: Profile - Todi Tinu-Constantin

6 Design Patterns

6.1 Facade Pattern

Implementat de: Todi Tinu-Constantin

- **Problema:** Gestionarea datelor unui utilizator este complexă, implicând interacțiunea cu mai multe subsisteme: sistemul de autentificare, sistemul de fișiere și baza de date relațională pentru informațiile biografice. Expunerea acestor subsisteme direct către interfața utilizatorului ar crea un cod greu de întreținut și predispus la erori.
- **Soluția:** Utilizarea șablonului Facade pentru a oferi o interfață simplificată și unificată. Profilul utilizatorului acționează ca o "fațadă" care maschează complexitatea din spate. Utilizatorul vede doar un formular simplu de editare.
- **Implementare:** S-a creat o structură de tip `ProfileViewModel` care agregă toate datele necesare. Controller-ul preia inputul simplu și orchestrează operațiunile complexe în fundal: salvarea imaginii pe disk, actualizarea tabeli `Profiles` și sincronizarea.

6.2 Observer Pattern

Implementat de: Popescu Florian

- **Problema:** Necesitatea de a disemina informații critice (locația stânelor și disponibilitatea produselor) către un număr mare de utilizatori simultan, decuplând logica de stocare a datelor de logica de vizualizare geografică. Modificarea stării unei stâne (ex: adăugarea unui produs) trebuie să fie reflectată vizual fără ca harta să știe detalii despre baza de date.
- **Soluția:** Implementarea mecanismului Observer unde harta acționează ca observator al modelului de date. Toți utilizatorii care accesează pagina principală devin "observatori" ai stânelor active.
- **Implementare:** Backend-ul furnizează lista de entități, iar biblioteca `Leaflet.js` (Observerul) subscrie la aceste date la încărcarea paginii. Harta reacționează prin randarea dinamică a markerilor pentru fiecare stână și produs, asigurând că toți utilizatorii vizualizează aceeași stare a sistemului.